Unidad 4: Programación Lógica

1. Introducción y Objetivos

La **programación lógica** es un paradigma de programación basado en la lógica formal, especialmente en la lógica matemática y de predicados .

A diferencia de los enfoques imperativos u orientados a objetos, en la programación lógica los problemas se formulan declarativamente mediante **hechos** y **reglas** que describen relaciones y conocimiento, en lugar de secuencias de pasos algorítmicos. Un programa lógico consiste esencialmente en una **base de conocimiento** (un conjunto de hechos que se suponen verdaderos) y un conjunto de reglas lógicas; a partir de ellos, un motor de inferencia puede **deducir nuevas conclusiones** aplicando las reglas de la lógica.

Este paradigma es el punto donde confluyen la lógica y la informática, pues utiliza el razonamiento lógico automatizado para inferir conclusiones a partir de datos. Por ejemplo, en lógica de primer orden podemos formalizar un razonamiento clásico: *"Todos los ingenieros son listos; Juan es ingeniero; por lo tanto, Juan es listo"*. En un programa lógico, este conocimiento se representaría mediante un hecho (`ingeniero(juan).`) y una regla general para cualquier persona (`listo(X) :- ingeniero(X).`), de modo que una **consulta** o pregunta al sistema (`?- listo(juan).`) permitiría inferir la nueva información de que Juan es listo basándose en la regla. Gracias a esta capacidad de representación y deducción, el paradigma lógico resulta especialmente útil en aplicaciones donde el **razonamiento formal** y la manipulación de relaciones complejas son centrales.

Importancia en la ingeniería: La programación lógica tiene importantes aplicaciones en múltiples campos de la ingeniería y la informática que requieren gestionar conocimiento y reglas complejas de decisión. Algunas áreas destacadas incluyen:

- **Sistemas basados en conocimiento (Sistemas Expertos)**: en dominios como la medicina, la ingeniería o el derecho, se construyen sistemas que emulan el razonamiento de expertos humanos mediante reglas lógicas, permitiendo diagnosticar fallos, recomendar soluciones o tomar decisiones fundamentadas en un conjunto complejo de conocimientos. Por ejemplo, un *sistema experto* en ingeniería eléctrica podría contener reglas para diagnosticar averías en un circuito a partir de síntomas observados.
- **Inteligencia Artificial y Lógica Difusa**: muchos métodos de IA simbólica utilizan lógica formal; la **ingeniería del conocimiento** se apoya en representación de conocimiento mediante hechos y reglas. Asimismo, extensiones como la lógica difusa permiten razonar con incertidumbre (utilizadas en control de sistemas, clasificación, etc.).
- **Procesamiento de Lenguaje Natural (PLN)**: la lógica se emplea para modelar gramáticas formales y extraer significado semántico. Por ejemplo, un analizador sintáctico puede usar reglas lógicas para identificar la estructura de una oración, o un sistema de chatbots puede basarse en hechos y reglas para entender y generar respuestas.
- **Bases de Datos Deductivas**: lenguajes lógicos como Datalog (que veremos más adelante) permiten realizar consultas complejas y deducciones sobre grandes conjuntos de datos, lo cual es útil en la gestión de bases de datos y sistemas de información. Un motor de base de

datos **deductiva** puede almacenar hechos (datos) y aplicar reglas lógicas para inferir datos derivados o responder consultas que implican relaciones implícitas.

- **Verificación y Comprobación de Modelos**: en la ingeniería de software y hardware, la lógica se usa para especificar formalmente propiedades de sistemas (por ejemplo, usando lógica temporal) y verificar automáticamente que un diseño cumple ciertas condiciones. Aunque esto va más allá de Prolog, comparte la idea de utilizar lógica para garantizar corrección (común en verificación de protocolos, circuitos, etc.).

Las ventajas del paradigma lógico, relevantes para la ingeniería, incluyen la **claridad declarativa** (se escribe *qué* se desea deducir, no *cómo* calcularlo), la **facilidad de modificación** (agregar un nuevo hecho o regla no altera las existentes, simplemente amplía el conocimiento), y la **cercanía con la lógica matemática** que facilita la validación formal de los programas. Esto último, al igual que en el lenguaje funcional, es fundamental en contextos de ingeniería donde la corrección y confiabilidad del software son primordiales.

Los objetivos de esta unidad son:

1. **Comprender los fundamentos de la lógica en computación** – Se explicarán los principios de la **lógica tradicional** (proposicional y de primer orden) y cómo estos sirven de base para representar conocimiento y razonar de forma automatizada.
2. **Conocer metodologías de ingeniería del conocimiento** – Se describirán metodologías empleadas para desarrollar **sistemas basados en conocimiento**, es decir, cómo pasar del conocimiento de un experto o de un problema del mundo real a una representación lógica formal y a una implementación ejecutable. Se hará hincapié en cada paso del proceso, ilustrándolo con ejemplos prácticos y código. Además, se mencionarán herramientas/metodologías clásicas como **GUPU** y su relevancia en la actualidad.
3. **Aprender el lenguaje Prolog** – Se introducirá el lenguaje de programación lógica *Prolog*, cubriendo su sintaxis, elementos clave (términos, predicados, reglas, consultas), operadores y estructuras de datos (incluyendo listas). Se proporcionarán ejemplos prácticos de código Prolog para resolver problemas sencillos, reforzando la comprensión de la semántica lógica.
4. **Explorar otros lenguajes lógicos** – Brevemente se presentarán otros lenguajes dentro del paradigma lógico, como **Answer Set Programming (ASP)** y **Datalog**, resaltando en qué se diferencian de Prolog y en qué tipos de aplicaciones se utilizan, para brindar una visión más amplia del campo.
5. **Desarrollar habilidades de resolución de problemas en Prolog** – A través de una serie de **ejercicios resueltos**, de dificultad incremental, el estudiante podrá ver la aplicación práctica de Prolog en diversos tipos de problemas. Cada ejercicio vendrá acompañado de una explicación detallada de la solución, para consolidar los conceptos aprendidos (desde consultas básicas y definiciones de reglas, hasta recursión y búsqueda de soluciones).

2. Metodologías

Una vez entendidos los fundamentos del paradigma lógico, es importante conocer cómo se aplica en la práctica, especialmente en el contexto de la **ingeniería del conocimiento**. En este capítulo abordaremos dos aspectos complementarios: primero, la **lógica tradicional** como base teórica para representar conocimiento; segundo, las **metodologías de desarrollo** empleadas para construir sistemas basados en conocimiento (incluyendo la mención de la herramienta educativa GUPU). A lo largo de la sección, detallaremos cada paso de una metodología típica con ejemplos prácticos y código ilustrativo.

2.1 Lógica tradicional

La **lógica tradicional** se refiere a los sistemas formales de lógica que han sido desarrollados por la matemática y la filosofía, principalmente la **lógica proposicional** y la **lógica de primer orden** (o lógica de predicados). Estos sistemas proveen la base teórica sobre la cual se construye la programación lógica. A continuación, resumimos los elementos clave de la lógica tradicional relevantes para la ingeniería del conocimiento:

- **Lógica proposicional**: modelo lógico donde las fórmulas (enunciados lógicos) no tienen estructura interna analizable, sino que se consideran proposiciones atómicas unidas por conectores lógicos. Por ejemplo, proposiciones como P: "El sensor está activado" o Q: "Se enciende la alarma", y reglas como $P \rightarrow Q$ (implicación: *si P entonces Q*). La lógica proposicional permite razonar sobre combinaciones de proposiciones (usando **AND**, **OR**, **NOT**, etc.) y derivar conclusiones lógicamente válidas. En ingeniería, puede servir para modelar sistemas simples de condiciones (e.g., circuitos lógicos, donde $P \wedge Q$ activa un resultado).
- **Lógica de primer orden**: extiende la lógica proposicional introduciendo la idea de **predicados** con argumentos y **cuantificadores** lógicos (*universal* $\forall$ y *existencial* $\exists$). Esto permite expresar relaciones entre objetos y propiedades individuales. Por ejemplo, podemos tener un predicado `ingeniero(X)` que afirma "X es ingeniero", y otro `listo(X)` que afirma "X es inteligente". Una fórmula de primer orden

  $$\forall X: \text{ingeniero}(X) \rightarrow \text{listo}(X)$$

  expresa *"para todo X, si X es ingeniero entonces X es listo"*. Combinado con un hecho singular como `ingeniero(juan)` (Juan es ingeniero), un **mecanismo de inferencia** puede deducir `listo(juan)` (Juan es listo) mediante la aplicación de la regla general. Este tipo de razonamiento, conocido como **modus ponens**, es fundamental en sistemas expertos: de una regla general y un hecho específico se obtiene una conclusión específica.

Los sistemas de programación lógica (como Prolog) se basan en la lógica de primer orden, pero suelen restringirse a una forma particular llamada **forma clausal**. En particular, usan **cláusulas de Horn**, que son fórmulas lógicas de la forma $A \leftarrow B1 \wedge B2 \wedge \cdots \wedge Bn$, donde A y cada B_i son literales atómicos (predicados posiblemente con variables). Esta notación corresponde directamente a la representación de **reglas** en Prolog: el literal A es la *cabeza* de la regla (lo que se quiere deducir) y B_1, ..., B_n conforman el *cuerpo* (las condiciones necesarias) Las cláusulas de Horn son lo

suficientemente expresivas para representar una gran variedad de conocimiento (implicaciones lógicas simples), a la vez que permiten algoritmos de inferencia eficientes (basados en **resolución** unificada y búsqueda).

En la lógica tradicional, la inferencia se puede llevar a cabo mediante **métodos de prueba** como el citado método de **resolución** (propuesto por J. A. Robinson en 1965) que sistematiza el proceso de deducción automática. Sin entrar en detalles formales, vale mencionar que Prolog implementa un algoritmo de inferencia similar a resolución sobre cláusulas de Horn: dado un objetivo (consulta), intenta unificarlo con la cabeza de alguna regla o hecho, y recursivamente satisface las condiciones del cuerpo de la regla, retrocediendo si alguna ruta de inferencia falla. Este mecanismo de *unificación y backtracking* (retroceso) es lo que permite a un programa lógico explorar las posibles conclusiones que se derivan de su base de conocimiento.

Conclusión de esta sección: La lógica tradicional proporciona el lenguaje y las herramientas formales para expresar conocimiento de forma declarativa. Para que dicho conocimiento resulte útil en ingeniería, debemos complementarlo con metodologías prácticas que nos guíen en cómo *adquirir, representar* e *implementar* el conocimiento de un dominio específico usando estas formas lógicas. A continuación, veremos cómo se estructura el proceso de construcción de un sistema basado en conocimiento, desde la captación del conocimiento experto hasta su codificación en un lenguaje lógico.

2.2 Metodologías en la Ingeniería del Conocimiento

La **Ingeniería del Conocimiento** es el área de la informática que se ocupa de la **construcción de sistemas basados en conocimiento** (SBC), también conocidos como sistemas expertos cuando emulan a especialistas humanos. Una metodología de ingeniería del conocimiento define un conjunto de fases o pasos sistemáticos para **transferir y formalizar el conocimiento** desde las fuentes (generalmente, expertos humanos o datos) hacia un sistema software. El objetivo es crear una **base de conocimiento** bien estructurada y efectiva que permita a un programa resolver problemas en un dominio específico utilizando reglas lógicas.

Aunque existen varias metodologías propuestas (por ejemplo, **CommonKADS** fue una metodología europea estándar para desarrollo de SBC en los 90s), la mayoría comparten etapas similares.

A continuación presentamos un proceso típico, dividido en pasos, para desarrollar una solución basada en el paradigma lógico.

Cada paso se ilustrará con un ejemplo práctico sencillo y fragmentos de código Prolog, para concretar cómo se podría llevar a cabo en la práctica. El ejemplo que usaremos es el desarrollo de un mini **sistema experto de diagnóstico** para una lámpara que no funciona (un problema cotidiano y simple, pero suficiente para ejemplificar las etapas):

Paso 1: Identificación del problema y del conocimiento relevante – En esta fase se define claramente cuál es el problema a resolver y qué tipo de conocimiento se necesita. Por ejemplo, **enunciado del problema**: *"La lámpara no enciende"*. Como ingenieros, debemos identificar las posibles causas de esta falla. Aquí la fuente de conocimiento podría ser un electricista o nuestro propio razonamiento técnico. Determinamos, por sentido común técnico, que las causas típicas de que una lámpara no funcione pueden ser: (a) que no esté enchufada, (b) que la bombilla (foco) esté

fundida, o (c) que haya una falla eléctrica más seria en el circuito. Este paso produce una lista de conceptos y relaciones relevantes: *lámpara, enchufada o no, bombilla fundida o no, corriente eléctrica*, etc., así como la formulación precisa del **objetivo**: diagnosticar por qué la lámpara no enciende.

Paso 2: Adquisición y recopilación del conocimiento – Implica extraer el conocimiento de las fuentes (expertos, documentación, experiencia) y organizarlo. Continuando con el ejemplo, recopilamos las reglas empíricas de diagnóstico:

- Regla C1: *"Si la lámpara no enciende y no está enchufada, entonces la causa es que está desenchufada"*.
- Regla C2: *"Si la lámpara no enciende, está enchufada y la bombilla está fundida, entonces la causa es la bombilla fundida"*.
- Regla C3: *"Si la lámpara no enciende, está enchufada y **no** está fundida la bombilla, entonces la causa es alguna falla eléctrica (cableado, circuito)"*.

Obsérvese que estamos obteniendo conocimiento en forma de reglas *si-entonces* (que luego podremos traducir a la forma lógica). Además de reglas, identificamos los **hechos o datos** que necesitaremos: por ejemplo, saber si la lámpara está enchufada o no, y si la bombilla está en buen estado o no, serán datos de entrada para el sistema. En sistemas más complejos, esta fase podría implicar entrevistas con expertos, revisión de manuales, o análisis de datos históricos para extraer patrones. El resultado de la fase 2 es un conjunto inicial de *hechos* y *reglas* en lenguaje natural o semiformal, que cubren el conocimiento del dominio.

Paso 3: Formalización (modelado conceptual y representación lógica) – Este paso consiste en llevar el conocimiento recopilado a una representación formal, no ambigua, apta para ser procesada por un programa. En nuestro caso, traducimos las reglas C1–C3 a **reglas lógicas** (predicados lógicos), definiendo los predicados necesarios. Primero, definimos cómo representaremos los *hechos*: por ejemplo, usaremos `enchufada(Lamp)` para indicar que una lámpara `Lamp` está enchufada a la corriente, y `fundida(Lamp)` para indicar que la bombilla de `Lamp` está quemada. También necesitaremos representar el síntoma "no enciende", que podemos llamar `no_enciende(Lamp)`. Finalmente, definiremos un predicado para la *causa diagnosticada*, por ejemplo `causa(Lamp, Causa)` donde `Causa` podría ser algo como `"desenchufada"`, `"bombilla"` o `"electrica"`. Con estos predicados, las reglas se formalizan así:

- C1 en lógica: $\forall$L: no_enciende(L)$\wedge\neg$enchufada(L)$\rightarrow$causa(L,"desenchufada").
- C2 en lógica: $\forall$L: no_enciende(L)$\wedge$enchufada(L)$\wedge$fundida(L)$\rightarrow$causa(L,"bombilla").
- C3 en lógica: $\forall$L: no_enciende(L)$\wedge$enchufada(L)$\wedge\neg$fundida(L)$\rightarrow$causa(L,"electrica").

Hemos usado aquí notación matemática con cuantificadores universales implícitos ("para todo Lámpara si... entonces..."). En la práctica, estas fórmulas lógicas se implementarán como *reglas de Horn* en Prolog. Nótese también que aparece la negación (por ejemplo ¬ enchufada(L)); en un programa lógico como Prolog utilizaremos la **negación como falla** (`\+ enchufada(L)`) para representar la falta de evidencia de que `L` está enchufada.

Paso 4: Implementación en un lenguaje lógico (codificación) – Llegados a este punto, se traduce el modelo formal a código ejecutable en el lenguaje escogido (usualmente Prolog para sistemas basados en lógica tradicional). Siguiendo el ejemplo, escribiremos un pequeño programa Prolog con

las reglas de diagnóstico y probaremos su funcionamiento. A continuación se muestra cómo quedarían nuestras reglas C1–C3 en Prolog, junto con algunos hechos de prueba:

```
% Hechos de ejemplo (situaciones conocidas):
no_enciende(lamp1).
enchufada(lamp1).      % lamp1 está enchufada
fundida(lamp1).        % lamp1 tiene la bombilla fundida

no_enciende(lamp2).
enchufada(lamp2).      % lamp2 enchufada
% (No declaramos fundida(lamp2), implicando que la bombilla NO
está fundida)

no_enciende(lamp3).
% (No declaramos enchufada(lamp3), implicando que NO está
enchufada)

% Reglas de diagnóstico:
causa(L, "desenchufada") :- no_enciende(L), \+ enchufada(L).
causa(L, "bombilla fundida") :- no_enciende(L), enchufada(L),
fundida(L).
causa(L, "falla electrica") :- no_enciende(L), enchufada(L), \+
fundida(L).
```

En este código, hemos incluido algunos **hechos** de ejemplo para tres lámparas (`lamp1`, `lamp2`, `lamp3`) que representan diferentes situaciones iniciales, y luego las **reglas** que implementan la lógica de diagnóstico explicada. Veamos en lenguaje natural qué expresa cada regla Prolog:

- La primera regla dice: "Para cualquier lámpara L, L tiene como causa 'desenchufada' **si** no enciende **y** no se puede demostrar que está enchufada". (En Prolog, `\+ enchufada(L)` significa que la premisa `enchufada(L)` falla, es decir, no hay hecho que indique que L está enchufada). Esta regla corresponde a la situación de que la lámpara esté desenchufada.
- La segunda regla: "L tiene como causa 'bombilla fundida' si no enciende, está enchufada **y** además su bombilla está fundida". Esto cubre la causa de la bombilla quemada.
- La tercera regla: "L tiene como causa 'falla electrica' si no enciende, está enchufada y **no** se puede demostrar que esté fundida". Esta última condición significa que la bombilla no está fundida (en nuestro programa, ausencia de hecho `fundida(L)` equivale a suponer que no está fundida). Entonces, estando enchufada y con bombilla buena, cualquier otro problema debe ser eléctrico (cableado, circuito).

Ahora nuestro programa lógico puede **inferir** la causa de la falla para distintos casos. Por ejemplo, si consultamos en Prolog: `?- causa(lamp1, C).` (¿cuál es la causa para `lamp1`?), el sistema intentará satisfacer el objetivo usando las reglas. En `lamp1` sabemos que `no_enciende(lamp1)`, `enchufada(lamp1)` y `fundida(lamp1)` son verdaderos. Al evaluar las reglas:

- Regla1: `no_enciende(lamp1)` es verdad, pero `\+ enchufada(lamp1)` es falso (porque `enchufada(lamp1)` es cierto), así que Regla1 no aplica.

- Regla2: `no_enciende(lamp1)` true, `enchufada(lamp1)` true, `fundida(lamp1)` true, por tanto se satisface la cabeza `causa(lamp1, "bombilla fundida")`. El sistema encontraría esta solución.
- Regla3 no se necesitaría evaluar una vez encontrada la regla2, pero conceptualmente: `no_enciende(lamp1)` true, `enchufada(lamp1)` true, `\+ fundida(lamp1)` **fallaría** porque `fundida(lamp1)` es cierto; por tanto Regla3 no aplica para lamp1.

El resultado de la consulta sería: `C = "bombilla fundida"`. En efecto, el programa diagnostica que para `lamp1` la causa es la bombilla fundida, acorde con los hechos que ingresamos.

Probando las otras situaciones de ejemplo: para `lamp2` (no enciende, enchufada, bombilla no fundida) la regla3 sería la que se cumple, produciendo `C = "falla electrica"`. Para `lamp3` (no enciende, no enchufada) aplicaría la regla1 y obtendríamos `C = "desenchufada"`. Así hemos implementado con éxito un pequeño sistema de diagnóstico usando programación lógica.

Paso 5: Verificación y validación – Una vez codificado el conocimiento, se debe comprobar que el sistema funciona correctamente, es completo y consistente. En sistemas expertos reales, esta fase es crucial: involucra probar muchos casos (ejemplos conocidos, casos de esquina) y verificar que las conclusiones del programa concuerdan con las que daría un experto humano. También implica revisar que no haya contradicciones en la base de conocimiento. En nuestro ejemplo, ya hicimos pruebas con lámparas lamp1, lamp2, lamp3. Si quisiéramos validar más, podríamos agregar un caso donde todo está correcto para comprobar que el sistema tal vez no infiere ninguna causa (por ejemplo, una lamp4 que enciende correctamente no tendría `no_enciende(lamp4)` en la base de conocimiento, por lo que ninguna regla deduciría causa, lo cual es deseado). Para un sistema más complejo, podríamos implicar al experto (el electricista) para confirmar que las reglas implementadas cubren todos los escenarios posibles y que las deducciones son válidas.

Paso 6: Mantenimiento y evolución – El conocimiento en muchos dominios no es estático: pueden aparecer nuevos casos, reglas o excepciones. La metodología contempla la actualización continua de la base de conocimiento. Por ejemplo, si en nuestro sistema de la lámpara descubriéramos otra causa posible (digamos, *"el interruptor está dañado"* como nueva causa), podríamos extender la base de conocimiento agregando una nueva regla (y hechos asociados, como un predicado `interruptor_malo(L)` si fuera comprobable). Una ventaja de la programación lógica es que su naturaleza declarativa facilita este mantenimiento: agregar una regla nueva no rompe la lógica de las existentes, simplemente agrega una vía adicional de deducción. Sin embargo, habrá que revalidar el sistema tras cambios importantes.

En resumen, la **metodología de ingeniería del conocimiento** guía el proceso desde el conocimiento informal hasta un sistema lógico operativo, asegurando rigor en cada paso: primero comprender y estructurar el problema, luego formalizarlo y codificarlo, y finalmente probarlo y refinarlo. Para problemas de mayor escala, existen metodologías más formales (como la mencionada CommonKADS) que descomponen el conocimiento en modelos (de tarea, de dominio, de inferencia, etc.) antes de llegar al código, pero la esencia es similar a este proceso simple que describimos.

Antes de cerrar esta sección, vale la pena **mencionar GUPU**, tal como se ha indicado. **GUPU** es un entorno educativo desarrollado a inicios de los 2000 para apoyar la enseñanza de la programación lógica (Prolog) de forma metódica. Se trata de un sistema interactivo que guía al estudiante por

todas las fases del desarrollo declarativo, desde la especificación del problema hasta la implementación y depuración del programa, proporcionando retroalimentación automática. GUPU ofrece características innovadoras como ejecuciones de referencia con diagnósticos, feedback inmediato de pruebas unitarias, herramientas de visualización de la ejecución y técnicas de *slicing* para localizar errores, todo ello integrado en un entorno unificado. En cierto modo, GUPU actúa como un tutor experto: verifica continuamente el progreso del alumno, marcando qué partes del problema ya fueron resueltas correctamente y cuáles requieren atención.

Aunque GUPU fue concebido para contextos académicos (para cursos de Prolog en la Universidad de Viena, por ejemplo), sus conceptos siguen siendo relevantes. Representa una **metodología asistida por herramienta**: enfatiza la importancia de una buena **especificación** (definir claramente la relación lógica que el programa debe capturar), promueve la **implementación incremental** (ir construyendo el programa mientras se comprueban casos de prueba) y facilita la **depuración declarativa** (encontrar errores basándose en las intenciones lógicas del programa más que en rastrear instrucciones). Si bien hoy en día GUPU no es ampliamente usado en la industria, la filosofía detrás de él —guiar el desarrollo de programas lógicos de manera estructurada y con retroalimentación— es valiosa en la ingeniería del conocimiento. En la práctica profesional, no es raro que se adopten metodologías análogas (por ejemplo, escribir primero casos de prueba que sirvan como especificación de comportamiento, algo parecido a *Test-Driven Development* adaptado a lógica, o usar depuradores de Prolog que ayudan a rastrear la unificación y el backtracking).

Las metodologías en la ingeniería del conocimiento proporcionan un *mapa de ruta* para construir sistemas basados en el paradigma lógico, asegurando que usamos la lógica tradicional de forma eficaz para modelar problemas reales. Herramientas como GUPU ejemplifican cómo apoyar didácticamente este proceso. En el siguiente capítulo nos enfocaremos en el lenguaje por excelencia de este paradigma, **Prolog**, profundizando en sus elementos y sintaxis, lo que nos permitirá materializar cualquier conjunto de hechos y reglas como un programa ejecutable.

3. El lenguaje Prolog

Prolog (cuyo nombre proviene de *Programmation en Logique*, francés para "programación en lógica") es el lenguaje de programación lógica más conocido y utilizado. Fue desarrollado a comienzos de la década de 1970 por Alain Colmerauer, Robert Kowalski y otros, y desde entonces ha sido aplicado extensamente en inteligencia artificial, sistemas expertos, procesamiento de lenguajes naturales y otras áreas. Prolog es un lenguaje **declarativo**: un programa Prolog se compone de declaraciones de hechos y reglas lógicas (cláusulas) que describen relaciones, y las respuestas se obtienen mediante un proceso de inferencia automática, no por ejecución de secuencias de comandos imperativos.

Formalmente, **Prolog está basado en la lógica de primer orden**, restringida a cláusulas de Horn. Cada sentencia en Prolog es una *cláusula* que puede ser de dos tipos: un **hecho** (cláusula sin cuerpo, simplemente una afirmación incondicional) o una **regla** (que tiene un cuerpo con condiciones). Además, las consultas que hacemos al programa corresponden a **objetivos lógicos** a demostrar a partir de los hechos y reglas. A continuación, se detallan los elementos principales del lenguaje Prolog: sus tipos de términos, sintaxis de predicados, uso de operadores y estructuras de datos, acompañados de ejemplos prácticos.

3.1 Términos básicos y sintaxis

En Prolog, **todo dato es un término**. Los términos pueden ser de diferentes tipos:

- **Átomos**: son constantes simbólicas, que representan objetos individuales o valores constantes. En la sintaxis de Prolog, un átomo se escribe típicamente comenzando con letra **minúscula** (o bien entre comillas simples si contiene caracteres especiales o empieza con mayúscula). Ejemplos de átomos: `juan`, `rosa`, `foo`, `'BuenosDias'` (átomo con mayúsculas y espacio, por eso entre comillas). Los átomos suelen denotar nombres de hechos o valores fijos, como nombres de personas, etiquetas, etc.
- **Números**: Prolog soporta números (enteros, reales de doble precisión) como un tipo especial de átomo numérico. Se escriben de la forma habitual: `0`, `42`, `3.14`, etc. Por ejemplo, `7` es un término numérico.
- **Variables**: representan *terminos desconocidos* o *por determinar.* Por convención sintáctica, en Prolog las variables se escriben comenzando con letra **mayúscula** o con guión bajo _. Ejemplos: `X`, `Persona`, `_temp`. Una variable en una regla Prolog es universalmente cuantificada (significa "para todo valor que la haga verdadera..."). Durante la resolución de una consulta, las variables se intentan *unificar* con átomos u otros términos para encontrar soluciones. Cabe destacar que el guión bajo solo (_) se usa para denotar una **variable anónima**, cuando su valor no nos interesa; cada ocurrencia de _ se trata como una variable distinta independiente.
- **Estructuras compuestas (functores)**: Son términos construidos mediante un **functor** (átomo) y una secuencia de uno o más términos como argumentos. Tienen la forma `functor(arg1, arg2, ..., argN)`. Las estructuras permiten crear objetos complejos jerárquicos. Por ejemplo, `padre(juan, maria)` es una estructura compuesta que podríamos interpretar como "Juan es padre de María". Aquí `padre` es el functor (y

típicamente se considera un **predicado** lógico) y `juan`, `maria` son sus dos argumentos. Otro ejemplo: una estructura para representar un punto en el plano podría ser `punto(3,5)` (coordenadas x=3, y=5). Las estructuras pueden anidarse: por ejemplo, `segundo(persona(juan), persona(maria))` – aunque este ejemplo es artificial, muestra que los argumentos a su vez pueden ser estructuras.

- **Listas**: Las listas son un tipo especial de estructura presente por conveniencia en Prolog, muy usadas para colecciones ordenadas de elementos. Se denotan con corchetes `[...]`. Por ejemplo, `[1,2,3]` es una lista de tres números; `[juan, maria, pepe]` es una lista de átomos (personas). Una lista vacía se escribe `[]`. Además, Prolog permite la deconstrucción de listas mediante la notación `[Head|Tail]`: por ejemplo, `[X|Y]` unifica con una lista cuya cabeza (primer elemento) es `X` y cuya cola (resto de elementos) es `Y` (que a su vez será una lista). Así, si tenemos la lista `[a,b,c]`, podría unificar con `Head = a` y `Tail = [b,c]`. Las listas internamente son estructuras encadenadas, pero el uso de la sintaxis especial las hace muy prácticas. Ejemplo: podemos representar un conjunto de hechos dinámicamente como una lista de átomos.

Predicados y cláusulas: Un **predicado** en Prolog es esencialmente un nombre que representa una relación o propiedad, y que puede tener varias definiciones (cláusulas). Cada *cláusula* es o bien un hecho o bien una regla para ese predicado. Por ejemplo, consideremos un predicado `progenitor/2` (la notación `/2` indica que el predicado tiene aridad 2, es decir, dos argumentos). Podemos definir varios **hechos** para `progenitor/2`:

```
progenitor(ana, beatriz).
progenitor(ana, carlos).
progenitor(beatriz, diana).
```

Estos hechos declaran quién es progenitor de quién (podemos interpretarlo como "Ana es madre de Beatriz", "Ana es madre de Carlos", "Beatriz es madre de Diana" por ejemplo). Aquí `progenitor` es el nombre del predicado, y cada par de argumentos concretos constituye un hecho distinto. También podemos definir reglas. Supongamos que queremos un predicado `abuelo(X, Z)` que sea verdadero si X es abuelo/abuela de Z. En lógica, X es abuelo de Z si existe un Y tal que X es progenitor de Y **y** Y es progenitor de Z. Eso se traduce a una regla Prolog:

```
abuelo(X, Z) :- progenitor(X, Y), progenitor(Y, Z).
```

Esta cláusula es una **regla**: tiene una cabeza (`abuelo(X, Z)`) y un cuerpo después del `:-`. El cuerpo contiene dos subobjetivos separados por coma, que en Prolog significa **conjunción** lógica (AND). Podemos leer la regla así: "X es abuelo de Z *si* X es progenitor de Y **y** Y es progenitor de Z". Las variables `X, Y, Z` quedan ligadas (unificadas) durante la búsqueda de soluciones: Prolog intentará encontrar algún `Y` que satisfaga ambos literales en el cuerpo, con X y Z dados o también variables según la consulta.

Notemos algunos aspectos sintácticos importantes ilustrados arriba:

- El **símbolo `:-`** separa la cabeza y el cuerpo de una regla, y se lee "si". A la izquierda de `:-` va lo que queremos deducir (conclusión) y a la derecha las condiciones.
- La **coma `,`** denota la conjunción lógica (AND), es decir, que deben cumplirse ambas condiciones.

- Podríamos usar **`; (punto y coma)`** para denotar disyunción (OR) en el cuerpo de una regla o en consultas, aunque a nivel de reglas usualmente es más claro escribir múltiples cláusulas separadas (ya que varias reglas con la misma cabeza funcionan como alternativas *o*).
- Cada cláusula termina con **punto `.`** al final.
- Los **nombres de predicados y functores** siguen las mismas reglas que los átomos (minúscula inicial normalmente). Ej.: `progenitor`, `abuelo` son predicados.
- Los **comentarios** en Prolog se introducen con `%` para comentarios de una línea, o `/* ... */` para bloques múltiples líneas.

Operadores incorporados y predicados especiales: Prolog, además de las estructuras básicas, provee ciertos operadores y predicados predefinidos útiles:

- **Unificación (`=`)**: La construcción `T1 = T2` intenta unificar (igualar) los términos `T1` y `T2`. Si la unificación tiene éxito, potencialmente liga variables. Por ejemplo, `X = 5` unifica X con 5 (X queda con valor 5). `punto(X, Y) = punto(3, 4)` unifica X=3, Y=4. Si la unificación falla (los términos no son compatibles), la consulta o rama de búsqueda falla.
- **Negación como fallo (`\+`)**: Ya mencionado, `\+ Goal` intenta probar `Goal`; si `Goal` tiene éxito, entonces `\+ Goal` falla, y si `Goal` falla, entonces `\+ Goal` succeed (tiene éxito). Representa una negación por falta de prueba (negación *débil*). Por ejemplo, en una regla usamos `\+ enchufada(L)` para significar "no se conoce/encontró hecho de que L esté enchufada", interpretado como L no está enchufada.
- **Corte (`!`)**: Es un operador especial llamado *cut*, que una vez alcanzado, le dice a Prolog que *fije* las decisiones tomadas hasta ese punto y no haga *backtracking* más allá de él. El cut se utiliza para optimización o para definir comportamientos deterministas, pero como es una construcción que afecta la procedimentalidad del programa (podando el espacio de búsqueda), se introduce con cuidado. En un nivel introductorio, es suficiente saber que `!` corta alternativas y que su mal uso puede hacer que se pierdan soluciones, por lo que solo se usa intencionadamente. (En nuestros ejemplos básicos probablemente no necesitaremos usar `!`).
- **Operadores aritméticos**: Prolog permite realizar cálculos aritméticos, pero de forma *declarativa*. Una particularidad es que la aritmética en Prolog requiere usar el predicado `is`. Por ejemplo, para evaluar una expresión y asignarla a una variable: `X is 2 + 3.` unificará X con 5. Si escribiéramos `X = 2 + 3.`, Prolog no calcularía 2+3 sino que unificaría X con la *estructura* `2+3` (ya que `+` puede verse como functor). Por eso, se usa `is` para forzar la evaluación. Además, hay operadores comparativos aritméticos: `=:=` (igualdad numérica tras evaluar ambos lados), `=\=` (diferencia numérica), `<, =<, >, >=` para menor, menor o igual, etc., que también evalúan expresiones aritméticas antes de comparar.
- **Operadores lógicos para metas**: Dentro de reglas o consultas, además de `,` (and) y `;` (or), existe `->` (implicación condicional; usado en construcciones if-then-else junto con `;`), pero estos se consideran características avanzadas del lenguaje Prolog como tal, más allá de la pura lógica de Horn. En este nivel introductorio, nos ceñiremos principalmente a hechos, reglas con `,` y el uso ocasional de `\+` y aritmética con `is`.

Veamos ahora un **ejemplo práctico completo** que ilustra varios de estos elementos en conjunto. Construiremos una pequeña base de conocimientos familiar y haremos algunas consultas:

```
% Hechos base de conocimiento familiar
progenitor(ana, beatriz).    % Ana es madre de Beatriz
progenitor(ana, carlos).     % Ana es madre de Carlos
progenitor(carlos, elena).   % Carlos es padre de Elena
progenitor(david, carlos).   % David es padre de Carlos
progenitor(david, fernando). % David es padre de Fernando

masculino(carlos).
masculino(david).
masculino(fernando).
femenino(ana).
femenino(beatriz).
femenino(elena).

% Regla: X es abuelo de Z si X es progenitor de Y y Y es
progenitor de Z.
abuelo(X, Z) :- progenitor(X, Y), progenitor(Y, Z).

% Regla: X es hermano de Y si comparten un progenitor Z y X != Y.
hermano(X, Y) :- progenitor(Z, X), progenitor(Z, Y), X \= Y.
```

En esta base de conocimiento:

- Se han definido hechos `progenitor/2` que establecen relaciones de padres e hijos entre varias personas (Ana, Beatriz, etc.). Observamos, por ejemplo, que `ana` es progenitor de `beatriz` y `carlos`, y `david` lo es de `carlos` y `fernando`.
- Hechos `masculino/1` y `femenino/1` determinan el género de algunas personas (por simplicidad, asumimos nombres diferentes implican individuos distintos).
- La regla `abuelo/2` captura la noción de abuelo/abuela: usa una variable intermedia `Y` que debe ser hijo de X y padre/madre de Z.
- La regla `hermano/2` define la hermandad: X y Y son hermanos si tienen al menos un progenitor en común (Z) **y** X no es igual a Y (`X \= Y` previene considerar a alguien hermano de sí mismo). Aquí usamos el operador `\=` que chequea desigualdad de términos.

Ahora podemos **consultar** esta base de conocimiento para obtener información derivada. Por ejemplo:

- Consulta 1: *"¿Quiénes son nietos de Ana?"* – En términos de Prolog, preguntamos por todos `Z` tales que `abuelo(ana, Z)` es cierto:

  ```
  ?- abuelo(ana, Z).
  ```

  Prolog intentará resolver `abuelo(ana, Z)` usando la regla. La cabeza de la regla unifica con `abuelo(ana, Z)` asignando `X=ana`. Luego las condiciones del cuerpo requieren encontrar un `Y` tal que `progenitor(ana, Y)` y `progenitor(Y, Z)` sean verdaderos. De `progenitor(ana, Y)` Prolog obtendrá posibles Y: `Y = beatriz` y `Y = carlos` (porque Ana es progenitora de Beatriz y de Carlos). Tomemos el primer: `Y=beatriz`. Ahora necesita `progenitor(beatriz, Z)`. Mirando los hechos, ¿Beatriz es progenitora de alguien? No (no hay hecho con beatriz como primer argumento), así que esa rama falla. Retrocede y prueba `Y = carlos`. Ahora busca

`progenitor(carlos, Z)`; sí hay un hecho `progenitor(carlos, elena)`, entonces unifica `Z = elena`. ¡Solución encontrada! Prolog reportará: `Z = elena.`. Además, buscará más soluciones (porque podría haber más nietos): tras esa, `Y = carlos` era la única opción válida que dio un Z. No hay más combinaciones, así que terminará. Resultado: **Elena** es nieta de Ana (lo deducimos vía Carlos).

- Consulta 2: *"¿David tiene alguna nieta?"* – Esto equivale a preguntar si existe un Z femenino tal que `abuelo(david, Z)` y `femenino(Z)`. Podríamos hacer:

  ```
  ?- abuelo(david, Z), femenino(Z).
  ```

  Prolog unirá ambas condiciones. Primero `abuelo(david, Z)` encuentra soluciones: David es progenitor de Carlos y Fernando; Carlos es progenitor de Elena, Fernando no es progenitor de nadie listado. Así, `Z = Elena` (vía `Y = Carlos`) es una solución de la primera parte. Luego `femenino(Z)` comprueba si Elena es femenino; según nuestros hechos, `femenino(elena)` es verdadero, por tanto la conjunción se satisface con `Z = elena`. Prolog devolvería `Z = elena` como solución (Elena es nieta de David y además es de género femenino). Podríamos consultar específicamente si Elena es nieta: `?- abuelo(david, elena).` que debería responder *sí* (true).
- Consulta 3: *"¿Quiénes son hermanos?"* – Podemos preguntar `?- hermano(X, Y).` sin instanciar, para que Prolog genere todos los pares X, Y que satisfacen la relación de hermandad. El sistema probará todos los hechos de progenitor para encontrar casos donde compartan progenitor. Por ejemplo, a partir de `progenitor(ana, beatriz)` y `progenitor(ana, carlos)`, encontrará solución `X=beatriz, Y=carlos` (y también `X=carlos, Y=beatriz` considerando la simetría implícita, ya que la regla no impone un orden en X e Y; cada par de hermanos aparecerá dos veces en respuestas de esta simple definición). Asimismo, de `progenitor(david, carlos)` y `progenitor(david, fernando)`, sacará `carlos` y `fernando` como hermanos. La condición `X \= Y` evitó casos triviales. El usuario podría filtrar duplicados o imponer un orden para evitar repeticiones, pero eso escapa a nuestro alcance inmediato. Lo importante es que Prolog encontrará todos los hechos implícitos de hermandad a partir de la base de progenitores.

Estos ejemplos muestran el **modo de operación de Prolog**: el intérprete recorre los hechos y reglas tratando de unificar el objetivo con cabezas de cláusulas, y desciende recursivamente por las condiciones (backtracking cuando necesario). Mientras existan posibilidades, explorará el espacio lógico de soluciones.

Una característica a resaltar es que **Prolog puede generar múltiples respuestas**. En las consultas interactivas, tras encontrar una solución, típicamente el intérprete espera a que el usuario pulse `;` para buscar la siguiente solución, o `ENTER` para terminar. En un programa embebido, se puede programar para recoger todas las soluciones usando predicados especiales (`findall/3`, `bagof/3`).

Estructuras de datos y recursión: Prolog maneja la recursión de forma natural para procesar estructuras como listas o para definir relaciones inductivas (ancestralidad, caminos, etc.). Por ejemplo, una clásica definición recursiva es la del **factorial** de un número:

```
factorial(0, 1).                % caso base: 0! = 1
factorial(N, F) :-
    N > 0,                      % premisa: N es positivo
    N1 is N - 1,                % calcula N1 = N-1
    factorial(N1, F1),          % llamada recursiva para el
factorial de N-1
    F is F1 * N.                % calcula F = F1 * N
```

Aquí vemos varios elementos juntos: dos cláusulas para `factorial/2`. La primera es un **hecho base** que estipula que el factorial de 0 es 1. La segunda es una **regla recursiva**: para calcular factorial(N, F), primero nos aseguramos de que N > 0 (utilizando un *guard* con el operador de comparación), luego decrementamos N (`N1 is N-1`), calculamos recursivamente el factorial de N1 obteniendo F1, y finalmente calculamos F = F1 * N. Usamos `is` para realizar las operaciones aritméticas. Con esta definición, si consultamos por ejemplo `?- factorial(5, X).`, Prolog encadenará llamadas recursivas hasta el caso base y nos dará `X = 120`. Este ejemplo muestra cómo Prolog mezcla la lógica (relación factorial definida inductivamente) con cálculos necesarios (resta y multiplicación) para obtener resultados numéricos.

Otro ejemplo, trabajando con listas, es el predicado **miembro** (`member/2`), que verifica si un elemento pertenece a una lista dada:

```
miembro(X, [X|_]).          % X es miembro si es la cabeza de la
lista
miembro(X, [_|T]) :-
    miembro(X, T).          % X es miembro si está en la cola
(recursivamente)
```

El primer hecho dice que X es miembro de una lista cuya cabeza es X (sin importar la cola, denotada por _). El segundo dice: X es miembro de una lista si no es la cabeza (la cabeza es anónima _ porque no nos interesa su valor), pero es miembro de la cola T. Esta definición recorre la lista elemento por elemento hasta encontrar el objetivo. Por ejemplo, consultar `miembro(carlos, [ana, carlos, elena])` satisfará la segunda cláusula ignorando `ana` y luego la primera cuando la cabeza sea `carlos`. En Prolog interactivo, también podríamos preguntar `?- miembro(X, [ana, carlos, elena]).` para generar los miembros de la lista; Prolog respondería `X = ana ; X = carlos ; X = elena.` sucesivamente al pedir más soluciones, mostrando cómo el backtracking encuentra *todas* las soluciones posibles.

Resumen de Prolog: Hemos visto la sintaxis básica (átomos, variables, estructuras, listas), cómo se definen predicados por hechos y reglas, y cómo funciona la inferencia mediante unificación, recursión y backtracking. Prolog exige una forma de pensar distinta: debemos configurar el problema en términos de relaciones lógicas y confiar en el motor de búsqueda para encontrar soluciones, lo cual contrasta con escribir pasos secuenciales. Este paradigma declarativo requiere práctica para dominarlo, pero una vez entendido, permite expresar soluciones de manera muy elegante y concisa para ciertos tipos de problemas (como veremos en los ejercicios). Antes de ejercitarnos en ello, cerraremos la exposición teórica mencionando brevemente otros lenguajes lógicos que complementan a Prolog en el ecosistema del paradigma lógico.

4. Otros lenguajes lógicos

Si bien Prolog es el lenguaje más representativo de la programación lógica, el paradigma lógico abarca otras variantes y lenguajes especializados que vale la pena conocer. A continuación presentamos brevemente dos de ellos: **Answer Set Programming (ASP)** y **Datalog**.

Answer Set Programming (ASP): La *Programación por Conjuntos de Respuestas* (traducción de *Answer Set Programming*) es un enfoque de programación lógica orientado a resolver problemas de búsqueda complejos, típicamente de naturaleza combinatoria (muchos de ellos NP-difíciles). ASP se basa en la semántica de **modelos estables** o *conjuntos respuesta* de la lógica: un programa ASP se compone de reglas (similares a cláusulas de Horn pero que pueden incluir negación más rica e incluso disyunciones en la cabeza) y su ejecución produce **conjuntos de literales** que representan *soluciones* del problema. En lugar de retornar soluciones vía unificación como Prolog, un *solver* de ASP calcula *todas* las combinaciones de hechos (un *modelo*) que satisfacen las reglas dadas.

En la práctica, para usar ASP se formulan los problemas declarativamente: por ejemplo, colorear un grafo, planificar rutas, resolver puzzles lógicos, etc., se codifican con reglas y restricciones. Luego se emplea un **solver de ASP** (como *clingo* o *DLV*) que genera los modelos estables (cada uno corresponde a una solución posible). A diferencia de Prolog, ASP realiza una búsqueda más orientada a *satisfacción global de restricciones* que a *deducción procedimental* – es más parecido a formular un SAT y obtener modelos. Un detalle es que ASP utiliza **negación fuerte** y **negación por ausencia** explícitamente, permitiendo expresar conocimiento **no monótono** (donde la ausencia de un hecho puede inferir otro, pero si más adelante se agrega el hecho ausente, la conclusión cambia). Esto lo hace apto para representar problemas de sentido común o con default rules.

Ejemplo: Un sencillo ejemplo de ASP podría ser un puzzle: *"En una fiesta hay 3 personas; cada persona o trae sombrero rojo o azul, y no puede haber dos personas con sombrero rojo."* En ASP escribiríamos reglas para asignar colores y una restricción que impide más de un rojo. El solver generaría todas las asignaciones válidas. ASP es utilizado en aplicaciones como planificación automatizada, configuración de sistemas, resolución de puzzles, y ha sido materia de competiciones internacionales donde diferentes solvers compiten en eficiencia. Para un programador lógico, ASP ofrece otra herramienta cuando se necesitan *todas* las soluciones o soluciones óptimas bajo ciertas condiciones, sin preocuparse del orden procedural de búsqueda de Prolog.

Datalog: Es un lenguaje lógico estrechamente relacionado con Prolog, pero enfocado principalmente en consultas y bases de datos deductivas. De hecho, Datalog puede verse como un *subconjunto de Prolog* con ciertas restricciones importantes:

- **Sintaxis y semántica:** Datalog utiliza reglas similares a Prolog (sin `:-` explícito a veces, dependiendo de la sintaxis, pero conceptualmente iguales: cabeza ← cuerpo). Sin embargo, **no permite funciones complejas como argumentos**, es decir, todos los términos deben ser o constantes o variables, lo que implica que Datalog *no* soporta estructuras anidadas arbitrariamente (esto asegura que el dominio de valores posibles sea finito y acotado por los hechos base).
- **No hay ejecución procedimental**: Datalog típicamente emplea un mecanismo de inferencia *bottom-up* en lugar del *top-down* de Prolog. Esto quiere decir que, en vez de responder a una

consulta mediante backtracking en las reglas, Datalog infiere exhaustivamente todos los hechos derivados posibles a partir de la base de conocimiento y luego responde consultas consultando ese conjunto inferido. Es como computar el *cierre transitivo* de las reglas. Esto se presta muy bien a optimizaciones en contexto de bases de datos (por ejemplo, se puede usar algoritmos de **evaluación seminaiva** para recomputar solo lo nuevo cuando se agrega conocimiento).

- **Aserciones de datos pero no modificaciones**: Datalog generalmente se enfoca en *consultas* a una base de datos fija; no suele incluir predicados extralogical para cortar, para hacer input/output o modificar dinámicamente la base, a diferencia de Prolog que sí tiene ciertas capacidades extralogicales. Esto lo hace más declarativo aún.
- **Uso en bases de datos y análisis**: Al ser acotado y más declarativo, Datalog ha sido implementado dentro de motores de bases de datos, permitiendo hacer consultas recursivas que SQL estándar no podía expresar fácilmente (como *alcanzabilidad en grafos*, jerarquías, etc.). También ha resurgido en áreas como análisis de programas (p.ej., hay herramientas que usan Datalog para especificar análisis estáticos de código, aprovechando que la evaluación Datalog puede ser altamente optimizada).

Un ejemplo de Datalog es casi idéntico a uno de Prolog, por ejemplo las reglas de ancestro (como las que definimos para `abuelo` pero más generales para cualquier ancestro) se escribirían igual. La diferencia es que en Datalog pedir *todas* las tuplas ancestro se hace naturalmente con la evaluación bottom-up. Muchos sistemas Datalog modernos extienden el lenguaje con *agregados* (conteos, mínimos, etc.) y otras características útiles para la analítica de datos.

En resumen, **ASP y Datalog** son dos caras distintas de la programación lógica declarativa: ASP está orientado a la resolución de problemas complejos mediante generación de soluciones (sistemas *solver* inteligentes), mientras que Datalog está orientado a **consultas lógicas optimizadas** típicamente sobre bases de datos o conjuntos de hechos muy grandes, garantizando terminación y decidibilidad (todo Datalog termina, porque no permite términos que crezcan indefinidamente, a diferencia de Prolog que es Turing-completo y puede no terminar en ciertos casos). Ambos lenguajes complementan el conocimiento de Prolog y amplían el horizonte de aplicaciones del paradigma lógico.

5. Ejercicios resueltos en Prolog

A continuación, presentaremos una serie de ejercicios prácticos **resueltos** en Prolog, con dificultad incremental. Cada ejercicio plantea un problema a resolver utilizando el paradigma lógico, seguido de su solución en código Prolog y una explicación detallada. Estos ejemplos sirven para afianzar los conceptos explicados y mostrar cómo aplicar Prolog en distintos contextos, desde consultas simples hasta problemas con recursividad y búsqueda de soluciones.

Ejercicio 1: Hechos y consultas básicas
Enunciado: Supongamos que queremos representar conocimiento sobre relaciones de **paternidad** (padre/madre e hijos) en una familia, y luego realizar consultas sobre esa información. En particular:

1. Definir algunos **hechos** Prolog que establezcan quién es padre o madre de quién (usaremos un predicado binario `padre/2` donde `padre(X,Y)` indica "X es padre/madre de Y"). Incluir al menos 5 relaciones.
2. Usando la base de conocimiento creada, realizar las siguientes consultas: (a) Preguntar si cierta persona es padre de otra (una consulta que dé un resultado booleano sí/no). (b) Obtener la lista de todos los hijos de una persona dada (consulta que encuentre todos los valores posibles para una variable).

Solución: Comenzamos definiendo los hechos. No es importante para Prolog distinguir padre de madre en este ejemplo, simplemente usamos `padre` de manera neutra para cualquier progenitor. Definiremos por ejemplo:

```
padre(juan, maria).
padre(juan, jose).
padre(maria, alberto).
padre(carlos, juan).
padre(carlos, elena).
```

Aquí tenemos 5 hechos:

- Juan es padre de María y de José.
- María es madre de Alberto.
- Carlos es padre de Juan y de Elena.

Con esa base de conocimiento, podemos consultarla. Veamos las consultas requeridas:

a) **Consulta booleana:** por ejemplo, preguntar `?- padre(juan, jose).` significa "*¿Juan es padre de José?*". El motor Prolog buscará un hecho que unifique con `padre(juan, jose)`. Efectivamente, hemos afirmado `padre(juan, jose).` en la base, por lo que la respuesta de Prolog será **`true`** (o "Sí"). En cambio, si preguntamos algo que no esté en la base, como `?- padre(maria, jose).` (¿María es madre de José?), Prolog no encontrará tal hecho y responderá **`false`** (o "No"), indicando que no puede deducir eso. En Prolog, cualquier consulta que no pueda demostrarse con los hechos y reglas dará false (recordemos la *asunción del mundo cerrado*: lo que no se conoce se considera falso).

b) **Consulta para obtener todos los hijos de una persona:** esto implica dejar la persona o el hijo como variable. Por ejemplo, "*¿Quiénes son los hijos de Juan?*". En Prolog, planteamos `?- padre(juan, Hijo).` donde `Hijo` es una variable. Prolog unificará `padre(juan, Hijo)` con todos los hechos existentes cuyo primer argumento sea `juan`. En nuestra base, las coincidencias son: `padre(juan, maria)` y `padre(juan, jose)`. Así, Prolog encontrará **dos soluciones**:

- Primero reportará `Hijo = maria`
- Y si solicitamos más (con `;`), `Hijo = jose`. Al no haber más hechos con `juan` como padre, al siguiente intento Prolog indicará que no hay más soluciones. En un entorno interactivo veríamos algo como:

```
?- padre(juan, Hijo).
Hijo = maria ;
Hijo = jose.
```

Esto muestra que los hijos de Juan son María y José, que concuerda con lo ingresado.

También podríamos consultar a la inversa: "*¿Quién es el padre de Juan?*": `?- padre(Padre, juan).` Prolog buscará quien tiene a juan como segundo argumento en un hecho `padre`. Encontrará `padre(carlos, juan)`, por lo que responderá `Padre = carlos.`.

En general, este ejercicio demuestra el uso básico de **hechos** y **consultas de un predicado**. Hemos visto cómo Prolog devuelve `true/false` para consultas completamente instanciadas (como preguntar por un hecho específico) y cómo devuelve valores para variables cuando se formulan consultas generales. También hemos observado la enumeración de múltiples soluciones gracias al backtracking.

Ejercicio 2: Reglas lógicas simples
Enunciado: Extendiendo el escenario familiar, definiremos una **regla** para inferir una relación a partir de los hechos de paternidad. Específicamente, queremos un predicado `abuelo/2` tal que `abuelo(X, Y)` sea verdadero si X es abuelo o abuela de Y. Luego, usando esa regla junto con los hechos de `padre/2` del ejercicio anterior, realizaremos una consulta para encontrar todos los abuelos existentes en la familia de ejemplo.

Solución: Como discutimos en la sección teórica, X es abuelo de Y si existe un Z tal que X es padre de Z **y** Z es padre de Y. En notación Prolog, la **regla** queda:

```
abuelo(X, Y) :- padre(X, Z), padre(Z, Y).
```

Incluimos esta regla en nuestra base de conocimiento junto a los hechos del ejercicio 1. Ahora, dada la base de ejemplo anterior, probemos la regla. Los hechos eran:

- Juan es padre de María y José.
- María es madre de Alberto.
- Carlos es padre de Juan y Elena.

De estos, podemos inferir abuelos:

- Carlos debería resultar abuelo de María y de José (porque Carlos es padre de Juan, quien a su vez es padre de María y José).

- Juan debería resultar abuelo de Alberto (Juan es padre de María, quien es madre de Alberto).
- Además, Carlos sería también abuelo de Elena? Verifiquemos: Carlos es padre de Juan, pero Juan no es padre de Elena (Carlos es padre directo de Elena, así que Carlos es el *padre* de Elena, no su abuelo).
- María sería abuela de alguien? María es madre de Alberto, pero no sabemos de hijos de Alberto en la base, así que no.

Hagamos la consulta general de todos los abuelos-nietos: `?- abuelo(Abuelo, Nieto).` Prolog examina la regla `abuelo(X, Y)` y buscará combinaciones de hechos `padre(X, Z)` y `padre(Z, Y)`. Siguiendo los hechos:

- Toma `padre(juan, maria)` (X=juan, Z=maria), luego busca `padre(maria, Y)`; encuentra `padre(maria, alberto)` (Y=alberto). Solución: `Abuelo = juan, Nieto = alberto`.
- Backtracking: aún con X=juan, toma `padre(juan, jose)`, busca `padre(jose, Y)`; no hay hecho con jose como padre, falla.
- Backtrack X: toma `padre(maria, alberto)` (X=maria, Z=alberto), busca `padre(alberto, Y)`; no hay, falla.
- Backtrack X: toma `padre(carlos, juan)` (X=carlos, Z=juan), busca `padre(juan, Y)`; encuentra Y=maria (y también Y=jose en otra iteración). Entonces soluciones: `Abuelo = carlos, Nieto = maria` y luego `Abuelo = carlos, Nieto = jose`.
- Backtrack X: `padre(carlos, elena)` (X=carlos, Z=elena), luego `padre(elena, Y)`; no hay, falla.
- Los demás hechos fallan similar al no encadenar dos niveles.

Prolog nos retornaría:

```
Abuelo = juan, Nieto = alberto ;
Abuelo = carlos, Nieto = maria ;
Abuelo = carlos, Nieto = jose.
```

(El orden puede variar, pero normalmente seguirá el orden de cláusulas y hechos como hemos analizado).

Verifiquemos con el razonamiento: Juan es abuelo de Alberto (correcto, vía María). Carlos es abuelo de María y de José (correcto, vía Juan). Coincide con nuestra deducción manual. No aparece María como abuela de alguien, ni Carlos como abuelo de Alberto (porque Carlos -> Juan -> Alberto no existe ya que Juan no es padre de Alberto, es María quien lo es). Todo concuerda.

Este ejercicio mostró cómo **añadir una regla** nos permite inferir nueva información de los hechos. Cabe destacar que no añadimos explícitamente quién es abuelo de quién, pero Prolog pudo derivarlo. Así, si mañana ampliamos la base con nuevos hechos de `padre`, la regla de abuelo automáticamente considerará esos casos. Por ejemplo, si agregáramos `padre(jose, monica).`, entonces Juan pasaría a ser abuelo de Mónica, y Carlos bisabuelo, etc., y nuestras consultas de `abuelo` reflejarían ese nuevo conocimiento sin cambiar la regla.

Ejercicio 3: Cálculo factorial (recursión numérica)
Enunciado: Implementar en Prolog el cálculo del **factorial** de un número natural. Recordemos que el factorial de N (denotado N!) se define como 1 si N=0, y si N>0 entonces N! = N * (N-1)! (el

producto de todos los enteros positivos hasta N). El objetivo es definir un predicado `factorial(N, F)` que sea verdadero cuando F es el factorial de N. Realizar pruebas con algunos valores (por ejemplo 5! = 120).

Solución: Tal como se mostró anteriormente, definimos dos cláusulas para el predicado `factorial/2`: una para el caso base (0) y otra recursiva para N > 0. En Prolog:

```
factorial(0, 1).                        % Caso base: 0! = 1
factorial(N, F) :-
    N > 0,                              % Aseguramos que N es
positivo
    N1 is N - 1,                        % Calculamos N1 = N-1
    factorial(N1, F1),                  % Llamada recursiva:
(N-1)!
    F is F1 * N.                        % Calculamos F =
(N-1)! * N
```

Explicación paso a paso: La primera línea es un **hecho** que cubre la definición de 0! = 1. La segunda es una **regla**; veamos cómo funciona cuando consultamos, por ejemplo, `factorial(3, F)`:

- Prolog verá que 3 no unifica con el hecho `factorial(0,1)`, así que intenta la regla. En la regla, N=3 unifica con N en la cabeza, dejando variable F para llenar.
- Dentro del cuerpo: `3 > 0` es verdad, luego `N1 is 3 - 1` calcula N1=2. Ahora Prolog necesita satisfacer `factorial(2, F1)`. Esto es una **nueva meta**, una sub-consulta que hará recursivamente.
- La recursión con N=2 sigue el mismo patrón: 2>0, calcula 1, luego llama `factorial(1, F2)`.
- Con N=1: 1>0, calcula 0, llama `factorial(0, F3)`.
- Con N=0: hace *match* con el caso base `factorial(0,1)`, por lo que devuelve `F3 = 1`.
- Retrocediendo: con N=1, ahora F2 = F3 * 1 = 1 * 1 = 1 (porque F3=1 y N=1), por la última línea de la regla. Así satisface `factorial(1,1)`.
- Retrocediendo: con N=2, ahora F1 = F2 * 2 = 1 * 2 = 2, satisface `factorial(2,2)`.
- Retrocediendo: con N=3, ahora F = F1 * 3 = 2 * 3 = 6, satisface `factorial(3,6)`.

Prolog devolverá `F = 6` para la consulta inicial. Si siguiéramos esta traza para `factorial(5, F)`, obtendríamos F = 120 como es esperado.

Pruebas: Consultando en Prolog:

```
?- factorial(5, X).
X = 120.
```

También podríamos consultar al revés: `?- factorial(4, 24).` para ver si Prolog confirma la verdad de 4! = 24. Efectivamente dará true. Una consulta interesante es dejar *ambos* argumentos como variables: `?- factorial(N, F).` En teoría factorial/2 define una relación entre N y F; sin embargo, debido a la naturaleza del predicado (y a que incluye aritmética), Prolog no generará infinitamente todos los pares N-F (porque la aritmética con `>` y `is` requiere N instanciado para hacer la resta). Si hacemos `?- factorial(N, F).`, Prolog esperará que N esté instanciado

para entrar en la regla recursiva (de lo contrario, no puede decidir `N > 0`). Por tanto esa consulta no generará soluciones porque la aridad requiere entrada en N. Esto ilustra que algunos predicados lógicos con aritmética se comportan esencialmente como *funciones direccionales*: aquí `factorial(N, F)` está pensado para calcular F dado N, y no al revés. Es importante tener en cuenta la dirección de uso de ciertos predicados debido a estos operadores no puramente lógicos.

Ejercicio 4: Operaciones con listas (miembro de una lista)
Enunciado: Definir un predicado `miembro(Elemento, Lista)` que determine si `Elemento` pertenece a la `Lista` dada. Debe funcionar tanto para verificar pertenencia (consultas del tipo "¿está X en la lista?") como para *generar* los elementos de la lista (consultas del tipo "dame todos los X que están en la lista"). Probar con una lista de ejemplo, incluyendo casos donde el elemento esté presente y no presente.

Solución: Utilizaremos una definición recursiva, tal como se explicó en la sección teórica, con dos cláusulas: una para el caso en que el elemento coincide con la cabeza de la lista, y otra para avanzar recursivamente sobre la cola de la lista. En Prolog:

```
miembro(X, [X|_]).         % X es miembro si es la cabeza de la
lista
miembro(X, [_|T]) :-
    miembro(X, T).         % X es miembro si es miembro de la cola
T
```

Explicación:

- La primera línea es un hecho (o regla sin cuerpo) que se lee: "*X es miembro de una lista cuyo primer elemento es X*". Los _ (guión bajo) indican "no nos importa lo que sigue". Es decir, sin importar lo que haya en la cola de la lista, si la cabeza es X, ya encontramos a X en la lista.
- La segunda línea es una regla que dice: "*X es miembro de una lista cuya cabeza es alguna cosa no relevante, y cuya cola es T,* ***si*** *X es miembro de T*". Aquí vemos la típica estructura recursiva que descarta el primer elemento de la lista (ya que no coincide con X, pues si coincidiera habría aplicado la primera regla) y sigue buscando en el resto.

Veamos cómo funciona en la práctica con ejemplos. Supongamos una lista de ejemplo: `[a, b, c]`.

- **Caso 1 (elemento presente)**: Consulta `?- miembro(b, [a, b, c]).`
  Prolog intenta unificar con la primera cláusula: `miembro(b, [X|_])`. Esto unifica X = `b` con la cabeza de la lista `[a|_]`? No, la cabeza real es `a`, así que `b` no unifica con `a`. Falla la primera cláusula.
  Entonces va a la segunda: `miembro(b, [_|T])`. Esto unifica con la lista `[a|T]`, es decir el primer elemento se ignora (_ toma el valor `a` pero no le asigna nombre) y `T` unifica con `[b, c]` (la cola). Ahora queda la meta del cuerpo: `miembro(b, T)` con `T=[b,c]`. Esta es una llamada recursiva. En esta llamada recursiva, aplica la primera cláusula: la cabeza de `[b,c]` es `b`, que coincide con el X buscado (`b`). ¡Éxito! Por lo tanto, la meta `miembro(b, [b,c])` resulta verdadera, lo que hace verdadera la meta original `miembro(b, [a,b,c])`. Prolog contesta **true**.

- **Caso 2 (elemento no presente)**: Consulta `?- miembro(x, [a, b, c]).` (buscando x en la lista).
  Primero intenta con la cabeza: ¿es `x` la cabeza de `[a,b,c]`? No (`a` != `x`).
  Reglas recursiva: parte la lista en _=`a`, `T`=`[b,c]`, y llama `miembro(x,[b,c])`.
  En esa recursión: ¿`x` es cabeza de `[b,c]`? no (`b` != `x`). Recurre de nuevo con _=`b`, `T`=`[c]`, llama `miembro(x,[c])`.
  Nueva recursión: ¿`x` es cabeza de `[c]`? no (`c` != `x`). Recurre con _=`c`, `T`=`[]`, llama `miembro(x, [])`.
  Ahora intentará las cláusulas con lista vacía `[]`. La primera cláusula `miembro(X, [X|_])` no unifica porque la lista no tiene cabeza (es vacía). La segunda `miembro(X, [_|T])` tampoco unifica porque la lista vacía no tiene forma `_ | T`. No hay más cláusulas, así que la llamada `miembro(x, [])` **falla**. Retrocede a la llamada anterior, que también falla, y así sucesivamente hacia arriba. Finalmente Prolog responde **false**, indicando que `x` no es miembro de la lista dada, lo cual es correcto.
- **Generar elementos de la lista**: Consulta `?- miembro(X, [a, b, c]).`
  Aquí X es variable libre y la lista `[a,b,c]` es concreta. Prolog intentará encontrar *todas* las posibilidades para X que satisfagan `miembro(X,[a,b,c])`.
  - Unificación con primera cláusula: `miembro(X,[X|_])` contra `miembro(X, [a,b,c])`. Esto unifica el X de la cabeza con la real cabeza `a`. Por lo tanto, se consigue una solución: `X = a`. Prolog la reporta. Pero esta cláusula también estableció que la cola _ puede ser cualquier cosa (en este caso `[b,c]`), pero como es no relevante, no nos impide continuar.
  - Backtracking para más soluciones: Ahora Prolog considera la segunda cláusula para la *misma* llamada inicial (o conceptualmente, la primera cláusula podría haber tenido más soluciones si la cabeza podía igualarse de otra forma, pero no es el caso; entonces pasa a la segunda). La segunda dice dividir la lista: `_ = a` y `T = [b,c]` y luego resolver `miembro(X, [b,c])`. Esta es una submeta nueva, recursiva.
  - En la submeta `miembro(X,[b,c])`, aplica de nuevo la primera cláusula: unifica X con `b`. Solución encontrada: `X = b`. Prolog retorna esa como segunda solución global.
  - Pide más: backtracking de nuevo, dentro de `miembro(X,[b,c])` usa la segunda cláusula: `_ = b`, `T=[c]`, llama `miembro(X,[c])`.
  - Ahí, primera cláusula da X = `c`. Retorna `X = c` como solución.
  - Pide más: dentro de `miembro(X,[c])`, segunda cláusula: `_ = c`, `T = []`, llamada `miembro(X,[])` falla, backtrack, no más soluciones en esa rama, retrocede todo, finalmente sin más alternativas.
  - Resultado: Prolog habrá listado `X = a ; X = b ; X = c`.

Como vemos, el predicado `miembro/2` funciona correctamente tanto para verificar pertenencia (casos true/false) como para iterar elementos. Este es un patrón común: la recursión sobre listas permite describir *traversal* (recorrido) de las listas de forma declarativa.

También cabe notar que `miembro/2` tal como está definido **puede generar respuestas repetidas si la lista tiene duplicados**. Por ejemplo, si la lista fuese `[a,b,a]` y preguntamos `miembro(a, [a,b,a])`, Prolog daría true (a está) y de hecho si pedimos todas las soluciones de miembro,

podríamos obtener `X = a ; X = b ; X = a` nuevamente, ya que recorrerá la lista y encontrará `a` dos veces. Esto es lógico dado el algoritmo.

Ejercicio 5: Caminos en un grafo (recursión con backtracking)
Enunciado: Consideremos un pequeño grafo (red de nodos conectados) representado mediante hechos de conexión entre nodos. Por ejemplo, hechos del tipo `conexion(ciudad1, ciudad2)` para indicar que hay un camino directo entre ciudad1 y ciudad2. Queremos definir un predicado `alcanzable(X, Y)` que sea verdadero si es posible ir desde X hasta Y *a través de una serie de conexiones*, no necesariamente directas (es decir, considerando caminos de varios tramos). Este predicado debe usar recursión para explorar la cadena de conexiones. Además, debemos tener cuidado con **evitar bucles** (por ejemplo, si A conecta a B y B conecta de vuelta a A, no queremos ciclo infinito). Encontrar todos los destinos alcanzables desde un nodo de origen dado mediante consultas.

Solución: Primero, definamos algunos hechos de ejemplo para el grafo. Usaremos nombres de ciudades como nodos:

```
conexion(madrid, valencia).
conexion(valencia, barcelona).
conexion(madrid, sevilla).
conexion(sevilla, granada).
conexion(barcelona, zaragoza).
conexion(valencia, mallorca).
```

Imaginemos este grafo dirigido (las conexiones las interpretaremos como unidireccionales tal como las escribimos). Tenemos:

- De Madrid se puede ir a Valencia y a Sevilla.
- De Valencia a Barcelona y a Mallorca.
- De Sevilla a Granada.
- De Barcelona a Zaragoza. No hemos explicitado las conexiones de vuelta (por ejemplo, no dijimos `valencia, madrid`), así que asumiremos que son de un solo sentido. También notemos que no hay ciclos obvios en este grafo; podríamos agregar alguno para probar, pero de momento así está bien.

Ahora definimos `alcanzable/2`. La idea es: Y es alcanzable desde X si:

- O bien hay una conexión directa de X a Y (caso base).
- O bien existe un nodo Z tal que X se conecta a Z **y** Y es alcanzable desde Z (recursividad).

Traducción Prolog básica:

```
alcanzable(X, Y) :-
    conexion(X, Y).
alcanzable(X, Y) :-
    conexion(X, Z),
    alcanzable(Z, Y).
```

Esta definición, sin más, lógicamente encuentra todos los Y alcanzables. Sin embargo, tal cual está puede entrar en bucles si el grafo tiene ciclos, porque no hay nada que evite volver a visitar nodos ya visitados. Una técnica típica para manejar eso es llevar un registro de visitados en un argumento

adicional o usar un predicado auxiliar. Para mantenerlo sencillo, asumiremos que el grafo de ejemplo no tiene ciclos problemáticos (en el dado, no los tiene, es más bien un árbol con rama adicional a Mallorca).

Explicación y pruebas: Probemos algunas consultas:

- Consulta: *"¿Es alcanzable Zaragoza desde Madrid?"* – `?- alcanzable(madrid, zaragoza).`
  Prolog intenta la primera regla: `conexion(madrid, zaragoza)` existe? No, no hay hecho así.
  Segunda regla: `conexion(madrid, Z), alcanzable(Z, zaragoza).`
    - Encuentra `conexion(madrid, valencia)` (Z = valencia), luego debe comprobar `alcanzable(valencia, zaragoza)`.
        - Para `alcanzable(valencia, zaragoza)`, prueba conexión directa: no hay `conexion(valencia, zaragoza)` dado directamente.
        - Recurre: `conexion(valencia, Z2), alcanzable(Z2, zaragoza)`. Encuentra `conexion(valencia, barcelona)` (Z2=barcelona). Ahora necesita `alcanzable(barcelona, zaragoza)`.
            - `conexion(barcelona, zaragoza)` existe como hecho directo, ¡éxito! Por la primera regla, `alcanzable(barcelona, zaragoza)` es verdadero.
            - Esto satisface la llamada recursiva anterior, por lo tanto `alcanzable(valencia, zaragoza)` es verdadero (vía Z2=barcelona).
            - A su vez, satisface la primera llamada: `alcanzable(madrid, zaragoza)` es verdadero (vía Z=valencia y luego barcelona). Prolog responderá **true**. Este encadenamiento corresponde al camino Madrid -> Valencia -> Barcelona -> Zaragoza, que efectivamente existe en nuestro grafo.
- Consulta: *"¿Qué destinos son alcanzables desde Madrid?"* – `?- alcanzable(madrid, Y).`
  Prolog debe encontrar todos los Y tales que la relación se cumpla. Siguiendo la definición:
    - Primero, usa la primera cláusula: `conexion(madrid, Y)`. Encontrará directamente `Y = valencia` y `Y = sevilla` (las dos conexiones directas). Estas son soluciones de `alcanzable/2` también, ya que conexión directa es un caso particular de alcanzable. Prolog enumerará: `Y = valencia ; Y = sevilla ; ...` y luego irá a la segunda cláusula.
    - Luego, la segunda cláusula: `conexion(madrid, Z), alcanzable(Z, Y)`. Esto abre sub-búsquedas:
        - Toma `Z = valencia` (por `conexion(madrid, valencia)`), y necesita `alcanzable(valencia, Y)`. Esa a su vez generará:
            - Directos de valencia: `Y = barcelona` y `Y = mallorca` (por `conexion(valencia, barcelona)` y

`conexion(valencia, mallorca)`). Entonces `alcanzable(valencia, barcelona)` y `alcanzable(valencia, mallorca)` son verdaderos, lo que produce soluciones para `alcanzable(madrid, Y)` heredadas: `Y = barcelona` y `Y = mallorca`.

- Recurriendo para valencia: `conexion(valencia, Z2), alcanzable(Z2, Y)`. Con Z2=barcelona, necesita `alcanzable(barcelona, Y)`.
  - Directo de barcelona: `Y = zaragoza`. Proporciona `Y = zaragoza` como alcanzable desde Madrid (lo mismo que encontramos antes pero ahora en modo enumeración).
  - `conexion(barcelona, Z3), alcanzable(Z3, Y)`: Barcelona solo tiene conexión a Zaragoza, ya usada; si no hay más (y no hay), termina esa rama.
- Z2=mallorca, verificar `alcanzable(mallorca, Y)`. Mallorca no tiene conexiones salientes en nuestros hechos, así que esa rama no produce nada más (solo los directos ya listados).

- La otra opción para Z en la primera recursion: `Z = sevilla` (por `conexion(madrid, sevilla)`), y chequeamos `alcanzable(sevilla, Y)`.
  - Directo de sevilla: `Y = granada` (porque `conexion(sevilla, granada)`). Da solución `Y = granada` para alcanzable desde Madrid.
  - Recurriendo: `conexion(sevilla, Z2), alcanzable(Z2, Y)`. Z2=granada, luego `alcanzable(granada, Y)`. Granada no tiene conexiones salientes definidas, así que no añade más.

- Recopilando soluciones únicas encontradas: `Y = valencia, sevilla, barcelona, mallorca, zaragoza, granada`. Prolog las iría dando en cierto orden: posiblemente:
  - `Y = valencia` (directo)
  - `Y = sevilla` (directo)
  - `Y = barcelona` (encontrado vía valencia)
  - `Y = mallorca` (vía valencia)
  - `Y = zaragoza` (vía valencia->barcelona)
  - `Y = granada` (vía sevilla) ... y luego no más.
- Notar: es posible que `Y = madrid` no salga como alcanzable desde madrid a menos que hubiera un ciclo (lo cual no hay y, en cualquier caso, debemos evitar considerarlo trivialmente alcanzable a sí mismo a menos que definamos reflexividad, lo cual no hicimos).

Esta consulta muestra cómo Prolog explora diferentes rutas mediante backtracking y recursión, generando todos los destinos alcanzables. En un grafo pequeño es manejable; en uno muy grande o con ciclos, se necesitarían las ya mencionadas precauciones de performance y corte de ciclos (por ejemplo, podríamos expandir la definición para que

`alcanzable(X,Y)` lleve un registro de nodos visitados: `alcanzable(X,Y,Visitados)` y chequee que Z no esté en Visitados antes de recursar, etc.).

Con este ejercicio, observamos el poder de Prolog para realizar **búsquedas exhaustivas** de soluciones en un espacio definido por reglas recursivas. El predicado `alcanzable/2` implementa una especie de algoritmo de recorrido de grafo (similar a *búsqueda en profundidad*). Prolog se encarga de la logística de la búsqueda (intentando caminos, retrocediendo en caso de callejones sin salida, etc.), mientras que nosotros solo le proporcionamos las relaciones básicas (conexiones directas y la regla recursiva de transitividad).

6. Conclusión

A lo largo de esta unidad hemos explorado el **paradigma lógico**, desde los fundamentos teóricos hasta ejemplos prácticos usando Prolog. Hemos visto que la programación lógica ofrece una manera declarativa de abordar problemas: en lugar de describir algoritmos paso a paso, uno describe *qué relaciones* existen y *qué condiciones* deben cumplirse, delegando al motor lógico la tarea de encontrar soluciones que satisfagan esas condiciones.

Destacamos la importancia del paradigma lógico en ingeniería del conocimiento y en áreas de la inteligencia artificial, subrayando cómo permite modelar el razonamiento humano de manera transparente. Mediante las **metodologías** de ingeniería del conocimiento aprendimos a estructurar el proceso de creación de sistemas basados en conocimiento: comenzando por la adquisición del conocimiento de expertos, formalizándolo en hechos y reglas lógicas, implementándolo (por ejemplo, en Prolog) y validándolo. También discutimos herramientas educativas como GUPU que reflejan buenas prácticas de desarrollo declarativo, enfatizando la relevancia de guiar al programador en especificar y depurar lógicamente.

Al introducir **Prolog**, cubrimos su sintaxis y semántica esencial: términos (átomos, variables, estructuras, listas) y la forma de expresar conocimiento a través de predicados, hechos y reglas. Observamos cómo Prolog ejecuta consultas mediante unificación y backtracking, lo que da lugar a un comportamiento emergente poderoso: enumeración automática de soluciones, deducción de información implícita, y manejo de búsquedas complejas con relativamente pocas líneas de código. Los ejemplos prácticos clarificaron estos conceptos, mostrando desde cómo consultar una base de datos de hechos simples hasta cómo definir recursivamente relaciones como ancestros, factoriales, pertenencia a listas o caminos en grafos.

También ampliamos la perspectiva mencionando brevemente otros lenguajes lógicos como **ASP** y **Datalog**, que si bien comparten el fundamento declarativo, atienden a necesidades específicas (problemas combinatorios con múltiples soluciones y consultas deductivas optimizadas, respectivamente). Esto nos recuerda que el paradigma lógico es rico y en evolución, incorporando enfoques como la *programación lógica con restricciones (CLP)*, la *programación lógica inductiva*, la *programación lógica probabilística*, etc., que extienden las capacidades básicas para enfrentar nuevos desafíos (como manejar incertidumbre, aprender reglas a partir de datos, o mezclar satisfactores de restricciones numéricas con lógica).

Desde una perspectiva de **ingeniería**, ¿cuáles son las virtudes y posibles limitaciones del paradigma lógico? En aplicaciones donde el conocimiento puede ser claramente expresado en forma de reglas lógicas (por ejemplo, sistemas expertos, configuradores, ciertas tareas de verificación y búsqueda), Prolog y lenguajes afines ofrecen un desarrollo rápido y una gran **transparencia**: las reglas son fáciles de leer y entender, casi como oraciones lógicas, facilitando el mantenimiento y la explicación del razonamiento (algo muy valioso en entornos críticos donde se necesita justificar cómo se llegó a una conclusión). Además, al separar el *qué* del *cómo*, el enfoque lógico permite probar diferentes estrategias de búsqueda o optimización sin cambiar la especificación del problema.

Por otro lado, en problemas donde dominan los cálculos secuenciales, la manipulación intensiva de estados cambiantes, o cuando el volumen de datos es enorme, la programación lógica pura puede no ser la opción más eficiente. A veces se combinan paradigmas: por ejemplo, sistemas que usan Prolog para la *lógica de alto nivel* (toma de decisiones, inferencia sobre reglas) pero delegan tareas numéricas o de bajo nivel a módulos imperativos eficientes (C, Python, etc.). Esto refleja una realidad de la ingeniería de software: no hay un único paradigma que domine en todo, sino que cada uno aporta fortalezas. El paradigma lógico aporta especialmente en el manejo de conocimiento, reglas y búsqueda inteligente.

Posibles aplicaciones futuras: Con el auge de la inteligencia artificial y la necesidad de sistemas cada vez más explicables, es posible que la programación lógica recobre protagonismo en combinación con otros enfoques. Por ejemplo, en el ámbito de la **IA explicable**, un sistema basado en reglas lógicas puede explicar exactamente por qué tomó cierta decisión (traza de inferencia), algo que los modelos de *deep learning* puro no pueden hacer fácilmente. Ya existen hibridaciones como neuro-simbolic AI, que combinan redes neuronales con representaciones lógicas para aprovechar lo mejor de ambos mundos (capacidad de aprendizaje automático con capacidad de razonamiento simbólico estructurado). Asimismo, el **Web Semántica** y las ontologías utilizan lógicas descriptivas (parientes de la lógica de primer orden) para razonar sobre conocimientos en la web – herramientas como OWL (Web Ontology Language) y razonadores asociados (Protege, Hermit, etc.) demuestran el uso práctico de la lógica en tecnologías modernas.

En sistemas de **planificación y robótica**, los planificadores basados en lógica (como STRIPS, PDDL, que se fundamentan en lógica) se usan para que robots o agentes automaticen secuencias de acciones cumpliendo objetivos. La **programación con restricciones** (por ejemplo en scheduling, asignación de recursos) extiende Prolog con capacidad de resolver restricciones aritméticas pesadas; estos se usan en la industria para resolver problemas de calendario, rutas, asignación de turnos, etc., de forma declarativa.

Como conclusión, el paradigma lógico es una pieza fundamental en el mosaico de paradigmas de programación y mantiene su vigencia en áreas especializadas de la ingeniería. Al aprender Prolog y los conceptos de la programación lógica, un ingeniero adquiere una forma distinta de pensar los problemas, enfocándose en las relaciones y condiciones más que en los pasos. Esta habilidad mental de abstracción lógica es valiosa incluso fuera de Prolog: mejora nuestra capacidad de formular problemas claramente (¿cuáles son mis datos? ¿qué quiero deducir de ellos?), algo aplicable en cualquier lenguaje o paradigma. Además, nos prepara para entender tecnologías de IA simbólica y para diseñar sistemas donde el conocimiento y su mantenimiento son centrales.